

Tech & Teach

Digital Chip Design Program

RTL Design

Task #09

Farah Alyahya

Rana Alsubaihi

Joud Almshari

Tala Alrsheed

Hoor Alkhedairi

Horiah Alqufaydi

Sandra Alomar

15/7/2026

Sequential Datapath Design Part 3

Version 1.0

Task #1.1

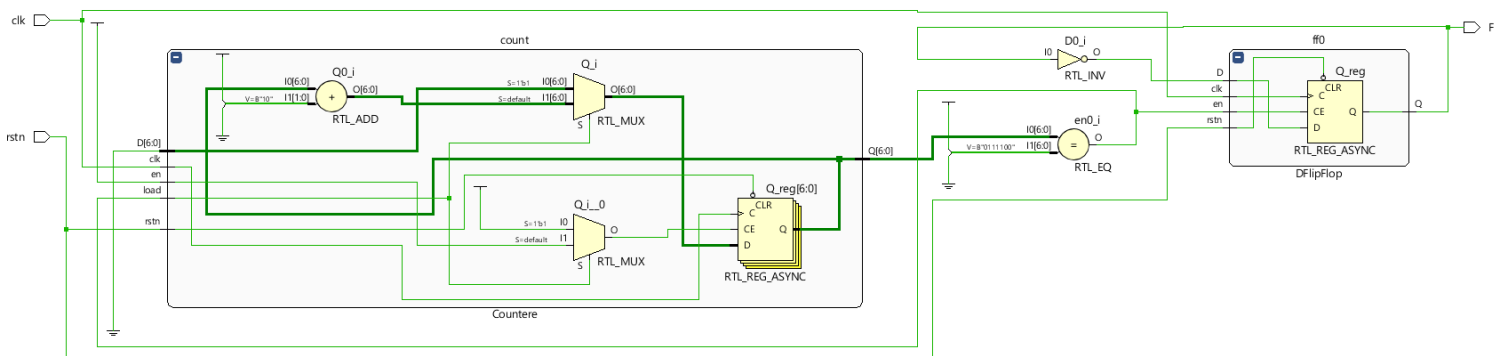
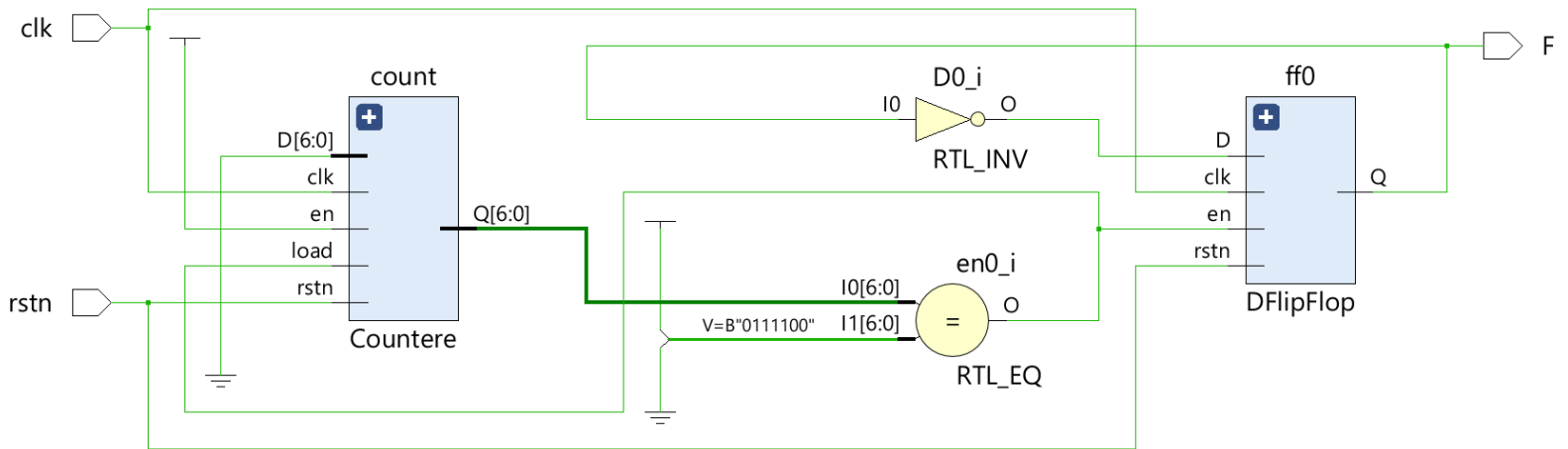
```
1 module Countere #(parameter N = 7) (  
2     input clk, rstn, en, load,  
3     input [N-1:0] D,  
4     output reg [N-1:0] Q  
5 );  
6     always_ff @(posedge clk, negedge rstn) begin  
7         if (!rstn)  
8             Q <= 0;  
9         else if (load)  
10            Q <= D;  
11        else if (en)  
12            Q <= Q + 2;  
13        else  
14            Q <= Q;  
15    end  
16 endmodule
```

```
1 module DFlipFlop (  
2     input clk, rstn, en, D,  
3     output reg Q  
4 );  
5     always_ff @(posedge clk, negedge rstn) begin  
6         if (!rstn)  
7             Q <= 0;  
8         else if (en)  
9             Q <= D;  
10        else  
11            Q <= Q;  
12    end  
13 endmodule
```

```

1 module TopDesign #(parameter N = 7) (
2     input clk, rstn,
3     output F
4 );
5     logic [N-1:0] w;
6
7     Countere #(N) count (
8         .clk(clk),
9         .rstn(rstn),
10        .en(1),
11        .load(w == 7'd60),
12        .D(7'd0),
13        .Q(w)
14    );
15
16    DFlipFlop ff0 (
17        .clk(clk),
18        .rstn(rstn),
19        .en(w == 7'd60),
20        .D(~F),
21        .Q(F)
22    );
23 endmodule

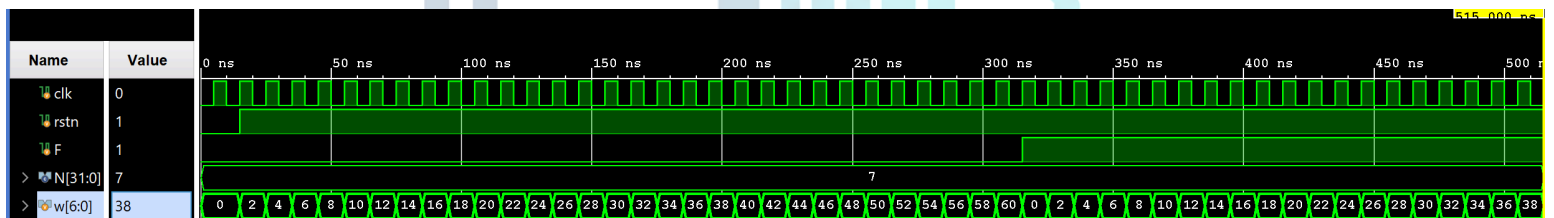
```



```

1  `timescale 1ns / 1ps
2  module TopDesign_tb;
3      parameter N = 7;
4      logic clk, rstn;
5      logic F;
6
7      TopDesign #(N) dut (
8          .clk(clk),
9          .rstn(rstn),
10         .F(F)
11     );
12
13     always begin
14         #5 clk = ~clk;
15     end
16
17     initial begin
18         clk = 0;
19         rstn = 0;
20         #15 rstn = 1;
21
22         #500 $finish;
23     end
24 endmodule

```



Tech & Teach

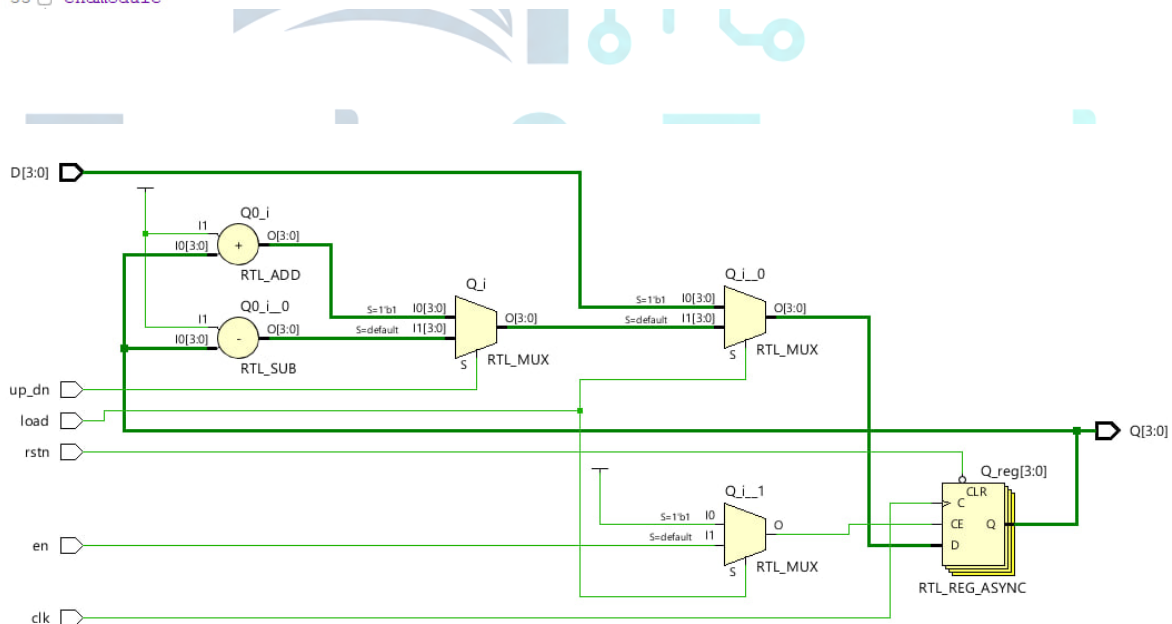
Task #1.2

C:/Users/farah/taskD9/taskD9.srscs/sources_1/new/taskD9.sv

```

1 module taskD9#(
2     parameter N = 4 // Default width is 4-bit, can be overridden during instantiation
3 ) (
4     input logic      clk,      // Clock signal
5     input logic      rstn,     // Asynchronous active low reset (1.2.1)
6     input logic      en,       // Synchronous active high enable (1.2.2)
7     input logic      load,     // Synchronous active high load (1.2.3)
8     input logic      up_dn,    // Direction control: 1 for UP, 0 for DOWN
9     input logic [N-1:0] D,     // N-bit parallel input data for loading
10    output logic [N-1:0] Q      // N-bit counter output
11 );
12
13 always_ff @(posedge clk or negedge rstn) begin
14     if (!rstn) begin
15         // 1.2.1: Asynchronous Active Low Reset
16         Q <= '0;
17     end else begin
18         if (load) begin
19             // 1.2.3: Synchronous Active High Load (Highest synchronous priority)
20             Q <= D;
21         end else if (en) begin
22             // 1.2.2: Synchronous Active High Enable
23             if (up_dn) begin
24                 Q <= Q + 1'b1; // Count UP
25             end else begin
26                 Q <= Q - 1'b1; // Count DOWN
27             end
28         end
29         // If en is 0 and load is 0, Q retains its current value
30     end
31 end
32
33 endmodule

```



C:/Users/farah/taskD9/taskD9.srscs/sim_1/new/taskD9_tb.sv

```

1  `timescale 1ns/1ps
2
3  module taskD9();
4      parameter N = 4;
5
6      logic          clk;
7      logic          rstn;
8      logic          en;
9      logic          load;
10     logic          up_dn;
11     logic [N-1:0]  D;
12     logic [N-1:0]  Q;
13
14     up_down_counter #(N) uut (.*) ;
15
16     always begin
17         clk = 0; #5;
18         clk = 1; #5;
19     end
20
21     initial begin
22         rstn = 0; en = 0; load = 0; up_dn = 1; D = 0;
23         #10 rstn = 1;
24
25         @(posedge clk);
26         D = 4'd10; load = 1;
27         @(posedge clk);
28         load = 0;
29
30         en = 1; up_dn = 1;
31         repeat (3) @(posedge clk);
32
33         up_dn = 0;
34
35         en = 1; up_dn = 1;
36         repeat (3) @(posedge clk);
37
38         up_dn = 0;
39         repeat (3) @(posedge clk);
40
41         en = 0;
42         repeat (2) @(posedge clk);
43
44         $finish;
45     end
46 endmodule

```

Task #1.3

```
dff_en sv x comparator sv x counter sv x clk_divider_top sv x tb_clk_divider sv x Untitled 2 x
C:/task_3/task_3.srcs/sources_1/new/dff_en sv
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
module dff_en (
    input logic clk,
    input logic rst_n,
    input logic en,
    input logic d,
    output logic q
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0;
        else if (en)
            q <= d;
        end
endmodule
endmodule
```

```
dff_en sv x comparator sv x counter sv x clk_divider_top sv x tb_clk_divider sv x Untitled 2 x
C:/task_3/task_3.srcs/sources_1/new/comparator sv
// Engineer:
//
// Create Date: 07/15/2026 05:16:33 PM
// Design Name:
// Module Name: comparator
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
module comparator #(
    parameter WIDTH = 8
) (
    input logic [WIDTH-1:0] a,
    input logic [WIDTH-1:0] b,
    output logic eq
);
    assign eq = (a == b);
endmodule
```

dff_en.sv × comparator.sv × counter.sv × clk_divider_top.sv × tb_clk_divider.sv × Untitled 2 ×

C:/task_3/task_3.srscs/sources_1/new/counter.sv

Q

↶

↷

✂

📄

📄

✖

//

📊

💡

⚙

15

//

16

// Revision:

17

// Revision 0.01 - File Created

18

// Additional Comments:

19

//

20

////////////////////////////////////

21

module counter #(
22 parameter WIDTH = 8
23) (
24 input logic clk,
25 input logic rst_n,
26 input logic en,
27 input logic load,
28 input logic [WIDTH-1:0] d,
29 output logic [WIDTH-1:0] q
30);
31 always_ff @(posedge clk or negedge rst_n) begin
32 if (!rst_n)
33 q <= '0;
34 else if (load)
35 q <= d;
36 else if (en)
37 q <= q + 1;
38 end
39 endmodule
40



Tech & Teach

Page 8 of 11

The screenshot shows a Verilog code editor with the following tabs: dff_en.sv, comparator.sv, counter.sv, clk_divider_top.sv*, tb_clk_divider.sv, and Untitled 2. The file path is C:/task_3/task_3.srscs/sources_1/new/clk_divider_top.sv. The code defines a module clk_divider_top with parameters N=100, inputs clk and rst_n, and output clk_div. It includes a counter module u_counter and a comparator module u_comp. The counter is configured with width WIDTH, clock clk, reset rst_n, enable 1'b1, load eq_sig, and initial value WIDTH{1'b0}. The comparator compares count_out with N{WIDTH-1:0} and outputs eq_sig. A DFF module u_dff is used to generate the output clk_div based on the clock clk, reset rst_n, enable eq_sig, and data ~clk_div.

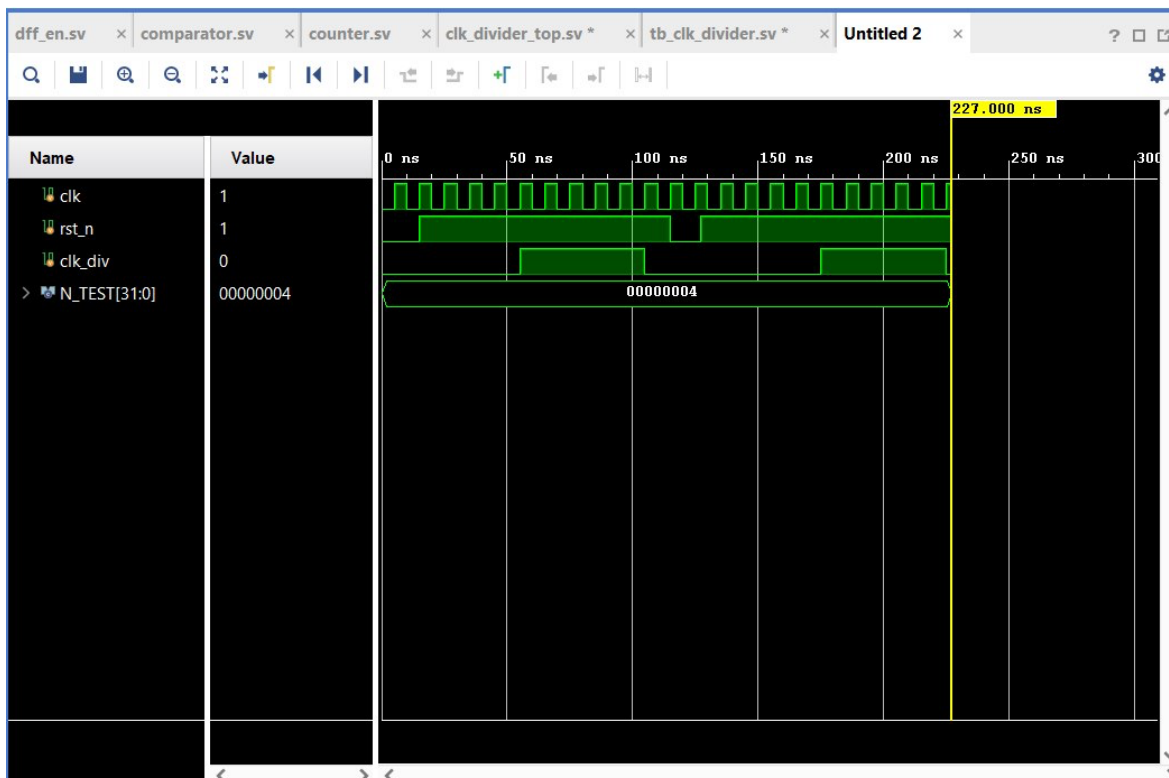
```
1  Save File (Ctrl+S) / 1ps
2
3  module clk_divider_top #(
4      parameter N = 100
5  ) (
6      input  logic clk,
7      input  logic rst_n,
8      output logic clk_div
9  );
10     localparam WIDTH = $clog2(N + 1);
11
12     logic [WIDTH-1:0] count_out;
13     logic             eq_sig;
14
15     counter #(
16         .WIDTH(WIDTH)
17     ) u_counter (
18         .clk(clk),
19         .rst_n(rst_n),
20         .en(1'b1),
21         .load(eq_sig),
22         .d({WIDTH{1'b0}}),
23         .q(count_out)
24     );
25
26     comparator #(
27         .WIDTH(WIDTH)
28     ) u_comp (
29         .a(count_out),
30         .b(N[WIDTH-1:0]),
31         .eq(eq_sig)
32     );
33
34     dff_en u_dff (
35         .clk(clk),
36         .rst_n(rst_n),
37         .en(eq_sig),
38         .d(~clk_div),
39         .q(clk_div)
40     );
41
42 endmodule
```



```

1  timescale 1ns / 1ps
2
3  module tb_clk_divider;
4
5      parameter N_TEST = 4;
6
7      logic clk;
8      logic rst_n;
9      logic clk_div;
10
11     clk_divider_top #(
12         .N(N_TEST)
13     ) uut (
14         .clk(clk),
15         .rst_n(rst_n),
16         .clk_div(clk_div)
17     );
18
19     initial begin
20         clk = 0;
21         forever #5 clk = ~clk;
22     end
23
24     initial begin
25         $display("Starting Simulation...");
26
27         rst_n = 0;
28         #15;
29         rst_n = 1;
30         $display("Reset released. Circuit should start dividing the clock.");
31
32         #100;
33
34         $display("Asserting Reset in the middle of operation to test stability...");
35         rst_n = 0;
36         #12;
37         rst_n = 1;
38
39         #100;
40
41         $display("Simulation Completed Successfully.");
42         $finish;
43     end
44
45     initial begin
46         $dumpfile("waves.vcd");
47         $dumpvars(0, tb_clk_divider);
48     end
49
50 endmodule

```



Work distribution

Section	Farah&Joud	Tala&Horiah	Rana&Hoor&Sandra
Report	✓	✓	✓
Task 1.1			✓
Task 1.2	✓		
Task 1.3		✓	